

FARMXPRESS

Soluciones agrícolas
a tu alcance



Manuel Jesús
Feria Santana

Proyecto 2ºDAW

ÍNDICE

1. Objetivo.....	4
2. Justificación.....	4
3. Tecnologías Usadas.....	5
3.1 Tecnologías Front-End.....	5
3.1.1 HTML.....	5
3.1.2 CSS.....	5
3.1.3 JavaScript.....	5
3.1.4 jQuery.....	5
3.1.5 AJAX.....	5
3.1.6 PDF24.....	5
3.2 Tecnologías Back-End.....	5
3.2.1 PHP.....	5
3.2.2 Stripe.....	5
3.3 Frameworks.....	6
3.3.1 Bootstrap.....	6
3.3.2 Agriculture	6
3.3.3 AdminLTE	6
3.4 Entornos de Desarrollo	6
3.4.1 Visual Studio Code	6
3.4.2 Modelo Vista Controlador (MVC)	6
4. Análisis	7
4.1 Diagrama de Casos de Uso	7
4.2 Diagramas de Secuencia	8
4.2.1 Insertar Alquiler	8
4.2.2 Insertar Pago	9
4.2.3 Insertar Reseña	10
4.2.4 Consultar Catálogo	11
4.2.5 Adquirir Suscripción	12
4.3 Diagrama Entidad-Relación	13
4.4 Diagrama de Tablas Relacionales	13
4.5 Tablas de la Base de Datos	14

4.6 Diagramas de Gant	18
4.6.1 Marzo – Abril	18
4.6.2 Abril – Mayo	19
4.6.3 Mayo – Junio	20
5. Guía de Estilos	21
5.1 Tipografía	21
5.2 Colores	21
5.3 Iconos	21
5.4 Imágenes	22
6. Desarrollo de la Aplicación	23
6.1 Programación del Lado Servidor	23
6.1.1 MVC (Modelo, Vista, Controlador)	23
6.1.2 Stripe – Pasarela de Pago	28
6.2 Programación del Lado Cliente	30
6.2.1 Validación Formulario de Registro	30
6.2.2 Validación Formulario de Producto	34
6.2.3 Formulario de Pago	34
6.2.4 Seguimiento de Productos	35
6.2.5 Generación de PDFs	36
6.2.6 Mensajes de Aviso Popup	37
7. Manual de Uso	40
7.1 Requisitos e Instalación	40
7.2 Usuario Invitado	40
7.3 Usuario Cliente	41
7.4 Usuario Proveedor	42
7.5 Administrador	42
8. Conclusiones	45
9. Proyección a Futuro	45
10. Enlaces Externos	46

1. OBJETIVO

El objetivo de esta aplicación es ayudar a los agricultores españoles facilitando una plataforma sencilla de usar en la que poder alquilar maquinaria agrícola de forma rápida y fácil, con la posibilidad de poder registrar a los proveedores de estas maquinarias que subirán fichas de sus productos a la propia web.

Los clientes podrán acceder a todo el catálogo de productos, marcar como favoritos los productos en los que estén interesados para su seguimiento, y dejar un comentario o reseña sobre los productos que hayan alquilado. Además, tanto clientes como proveedores podrán adquirir suscripciones con las que disfrutar de ciertas ventajas.

Por otro lado, el sitio web contará con una zona de administrador en la que poder gestionar usuarios, categorías, productos y las diferentes suscripciones.

2. JUSTIFICACIÓN

La creación de una web de renting de maquinaria agrícola se justifica por la ausencia de una plataforma centralizada y abierta que permita a múltiples empresas proveedoras ofrecer sus productos para alquiler.

Actualmente, no existe una página que funcione como marketplace especializado en maquinaria agrícola, donde diferentes proveedores puedan subir sus propias máquinas y los usuarios clientes puedan comparar, elegir y alquilar según sus necesidades. La mayoría de las webs disponibles están enfocadas en el renting de vehículos, como coches o furgonetas, lo cual deja desatendido un sector clave como el agrícola, que requiere soluciones específicas para sus tareas.

Además, la única plataforma existente centrada en el renting de maquinaria agrícola pertenece a John Deere y se limita exclusivamente a los productos de su marca, lo que reduce significativamente la variedad de opciones disponibles para los usuarios. Esta limitación crea una barrera tanto para clientes que buscan alternativas más asequibles o específicas, como para otras empresas proveedoras que desean participar en el mercado digital del alquiler. Por tanto, una web abierta, multisectorial y enfocada exclusivamente al alquiler de maquinaria agrícola cubriría una necesidad real, fomentaría la competencia, daría visibilidad a más marcas y facilitaría el acceso a herramientas modernas para pequeños y medianos agricultores.

3. TECNOLOGÍAS USADAS

3.1 TECNOLOGÍAS FRONT-END

3.1.1 HTML (HYPERTEXT MARKUP LANGUAGE): El lenguaje de marcas estándar para estructurar y organizar el contenido de una página web. Define elementos como encabezados, contenedores, textos y formularios.

3.1.2. CSS (CASCADING STYLE SHEETS): Lenguaje de estilos en cascada que define cómo se presenta y se diseña el contenido estructurado por HTML. Controla colores, fuentes, márgenes, tamaños... de los elementos en la página.

3.1.3. JS (JAVASCRIPT): Es un lenguaje de programación de lado cliente utilizado para agregar dinamismo y funcionalidad avanzada a las páginas web, como validaciones, animaciones y actualizaciones en tiempo real.

3.1.4. JQUERY: Es una biblioteca de JavaScript que simplifica el manejo del DOM, la manipulación de eventos, animaciones y peticiones AJAX.

3.1.5. AJAX (ASYNCHRONOUS JAVASCRIPT AND XML): Es una técnica que combina JavaScript con el uso de objetos XMLHttpRequest para realizar peticiones al servidor de forma asíncrona, como insertar o actualizar la base de datos.

4.1.6. PDF24: Una librería para JavaScript que permite la generación de archivos PDF de forma sencilla.

3.2 TECNOLOGÍAS BACK-END

3.2.1 PHP (HYPERTEXT PREPROCESSOR): Es uno de los lenguajes de programación del lado del servidor más populares, diseñado para crear aplicaciones web dinámicas. Se utiliza para procesar formularios, interactuar con bases de datos, generar contenido HTML dinámico y manejar sesiones de usuario.

3.2.2 PASARELA DE PAGO STRIPE: Un cliente oficial para trabajar con la API de Stripe. Facilita la integración de pagos, suscripciones y otros servicios financieros en aplicaciones PHP. Permite, además, simular la acción de pagos para comprobar su funcionamiento.

3.3 FRAMEWORKS

3.3.1 BOOTSTRAP: Es un framework de front-end que facilita la creación de sitios web responsivos y modernos. Proporciona un sistema de rejillas, componentes predefinidos (botones, formularios, barras de navegación, etc.) y utilidades CSS y JS para agilizar el diseño y desarrollo.

3.3.2 AGRICULTURE: Una plantilla creada mediante Bootstrap que cuya temática principal está relacionada con la agricultura y el campo, presentando el verde como color principal

<https://bootstrapmade.com/agriculture-bootstrap-website-template/>

3.3.3 ADMINLTE: Otra plantilla gratuita creada mediante Bootstrap. Proporciona componentes predefinidos, y varias plantillas con diferentes estilos para desarrollar rápidamente paneles administrativos.

3.4 ENTORNOS DE DESARROLLO

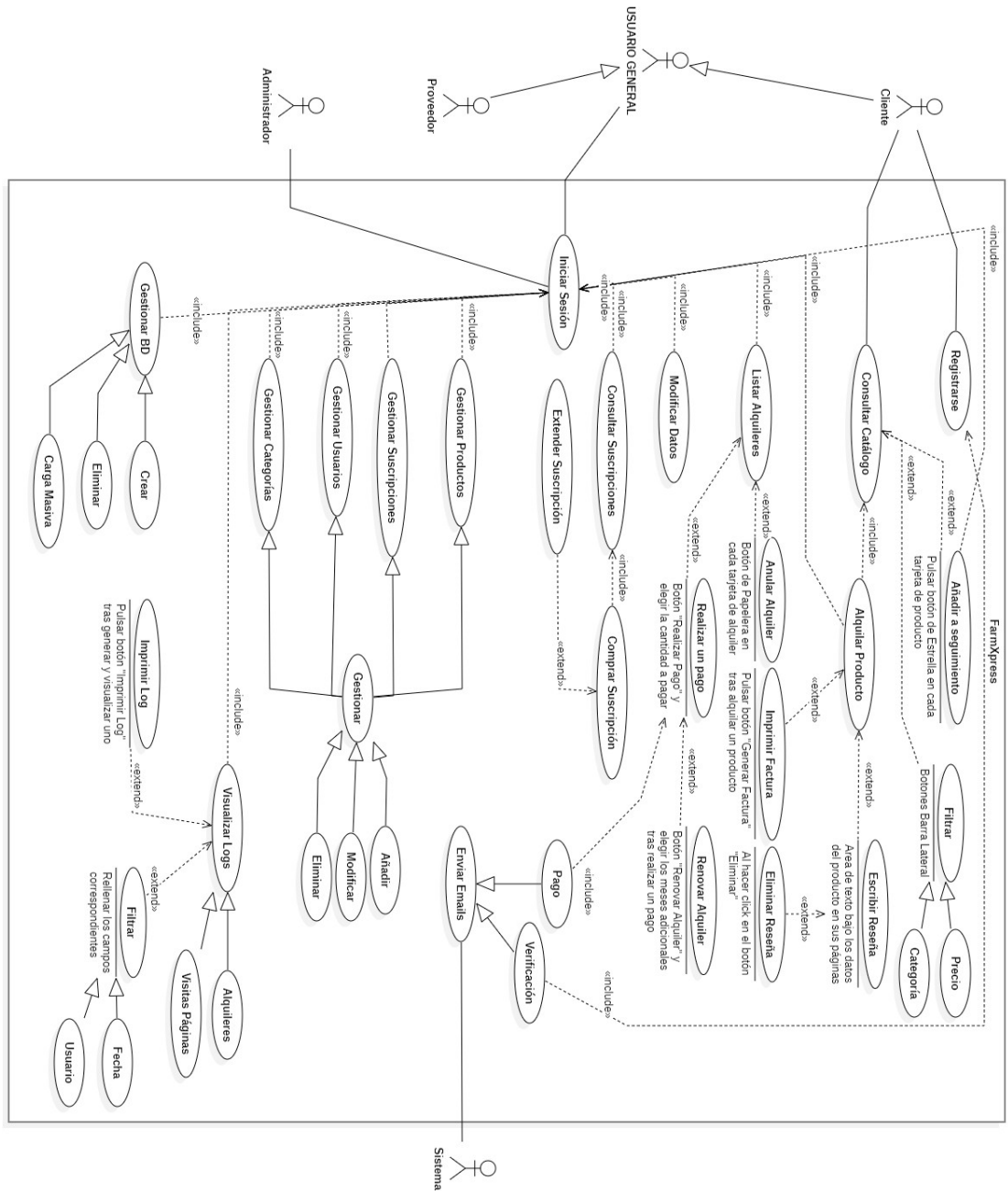
3.4.1 VISUAL STUDIO CODE (VSCODE): Es un editor de código fuente ligero y de código abierto desarrollado por Microsoft. Ofrece soporte para varios lenguajes de programación y una amplia gama de extensiones, como depuración, control de versiones y completado de código.

3.4.1 MODELO VISTA CONTROLADOR (MVC): Es un patrón de diseño de software utilizado en el desarrollo de aplicaciones web. Se divide en tres componentes:

- **Model (Modelo):** Maneja la lógica de los datos y la interacción con la base de datos.
- **View (Vista):** Se encarga de la presentación de los datos y la interfaz de usuario.
- **Controller (Controlador):** Gestiona la entrada del usuario y actualiza el modelo y la vista según sea necesario.

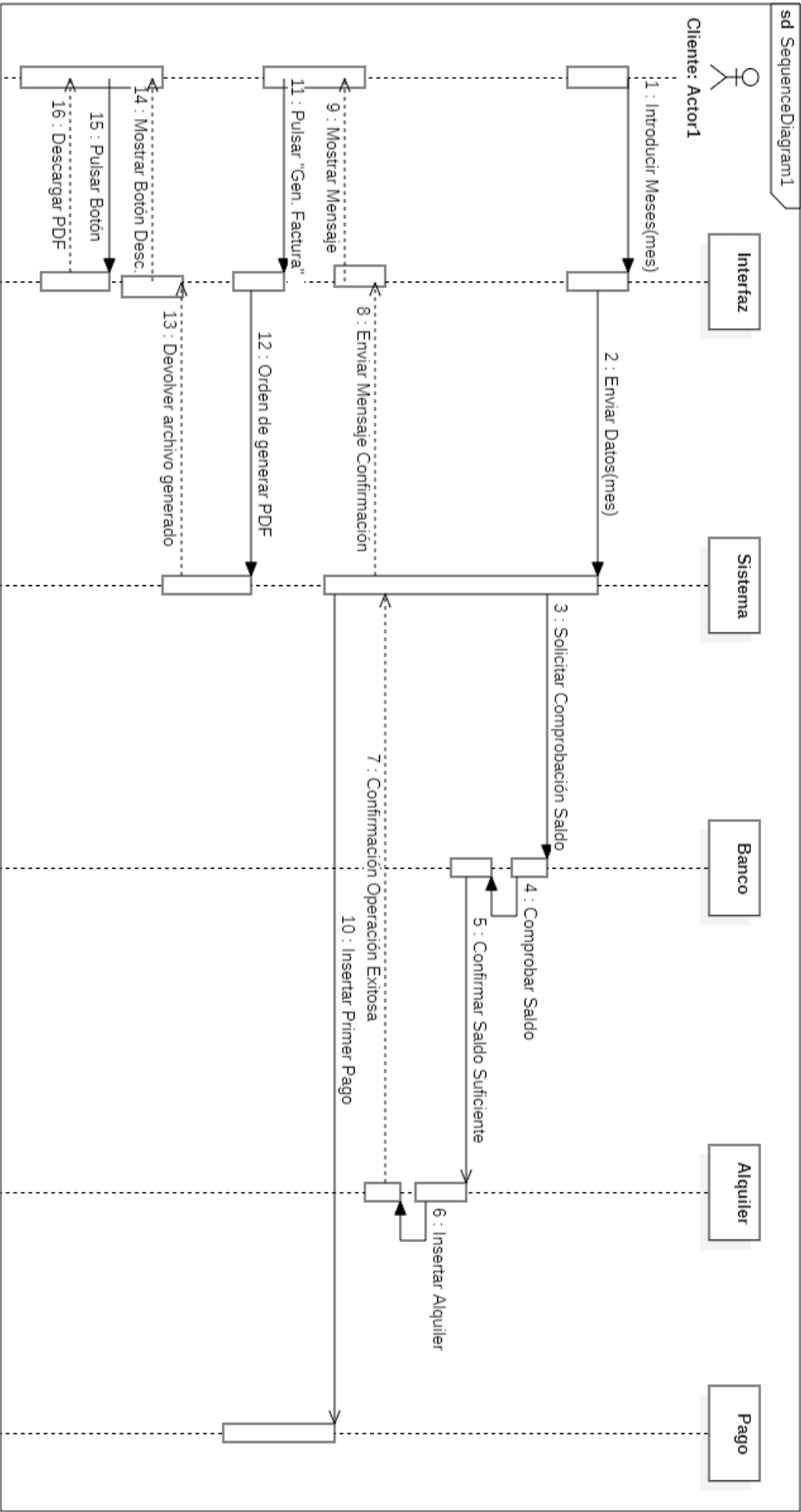
4. ANÁLISIS

4.1 DIAGRAMA DE CASOS DE USO

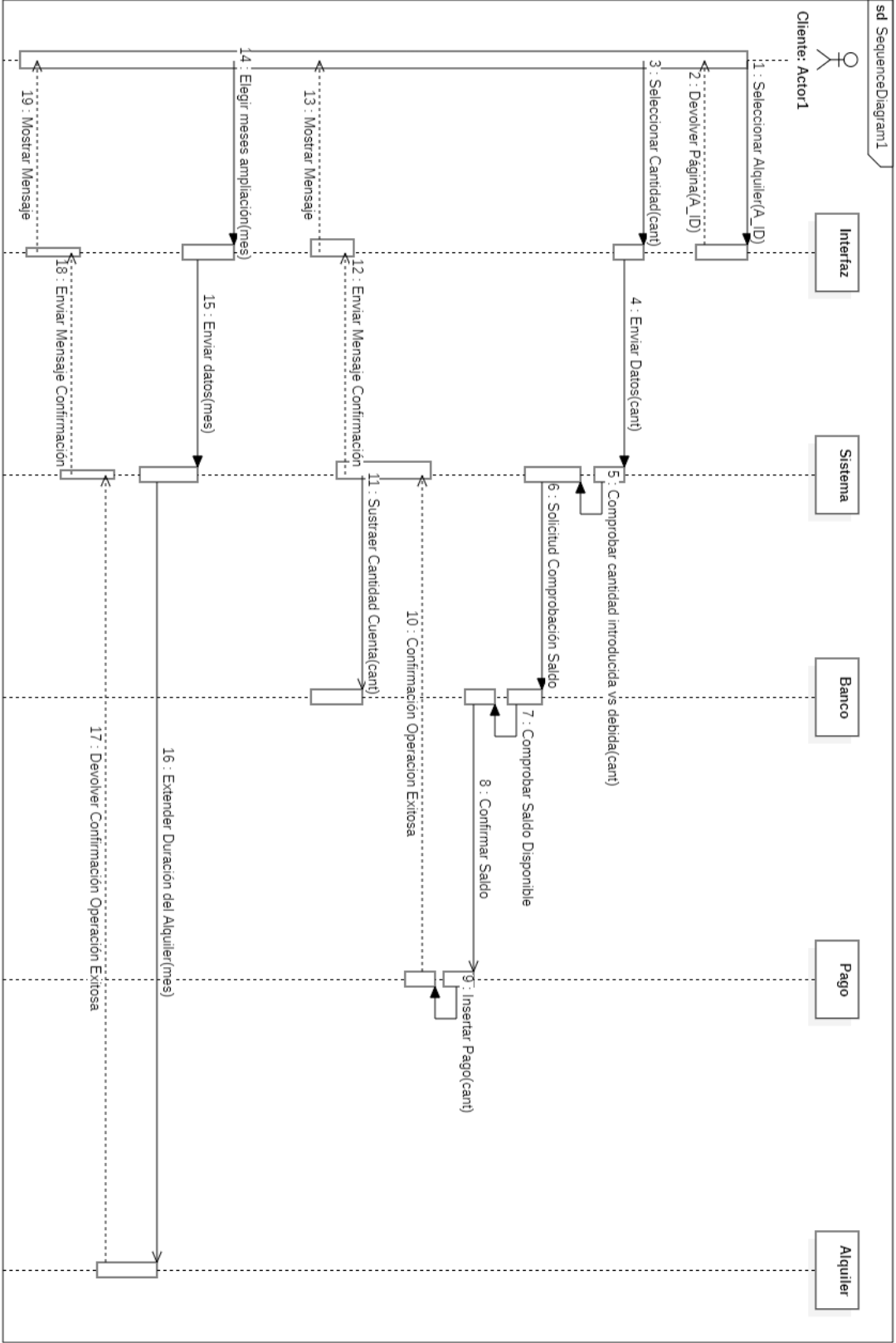


4.2 DIAGRAMAS DE SECUENCIA

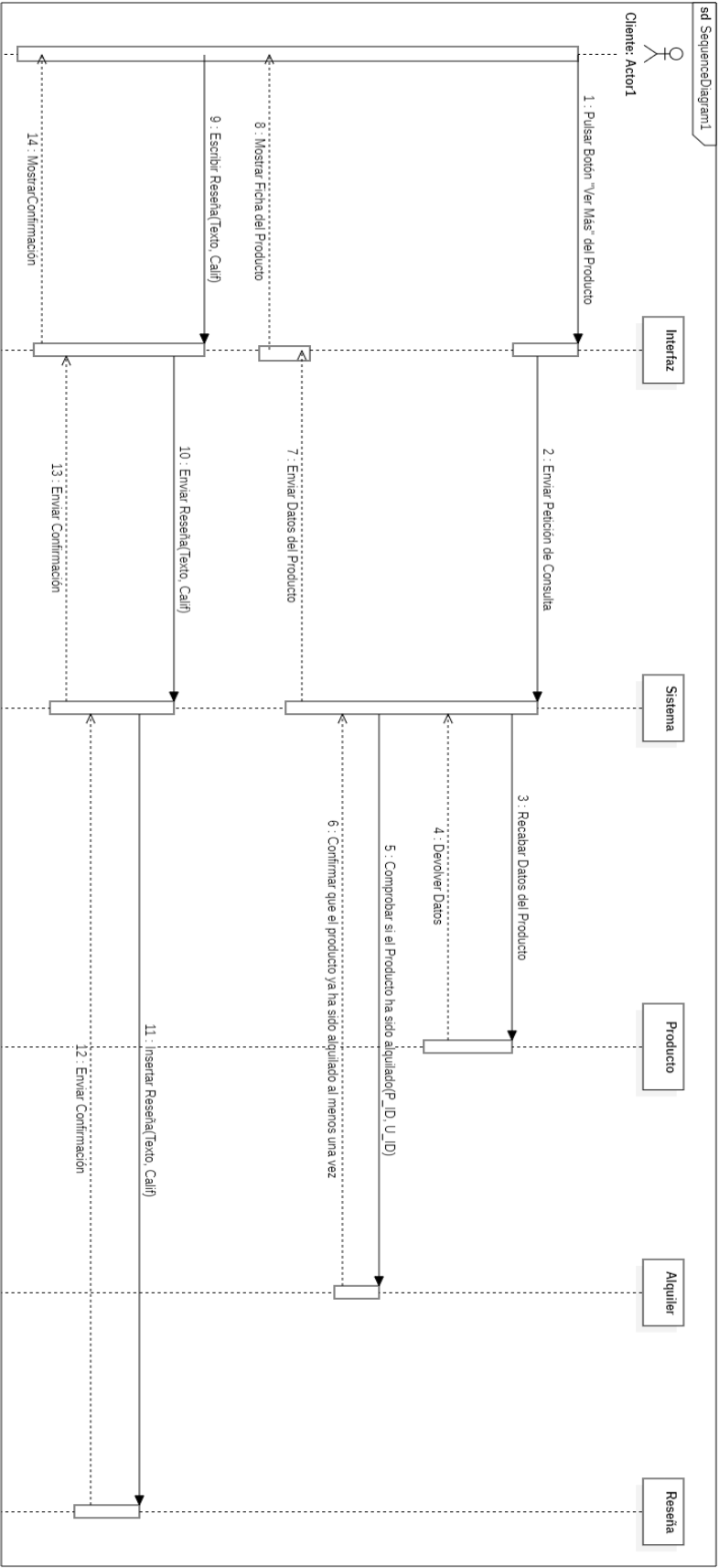
4.2.1 INSERTAR ALQUILER



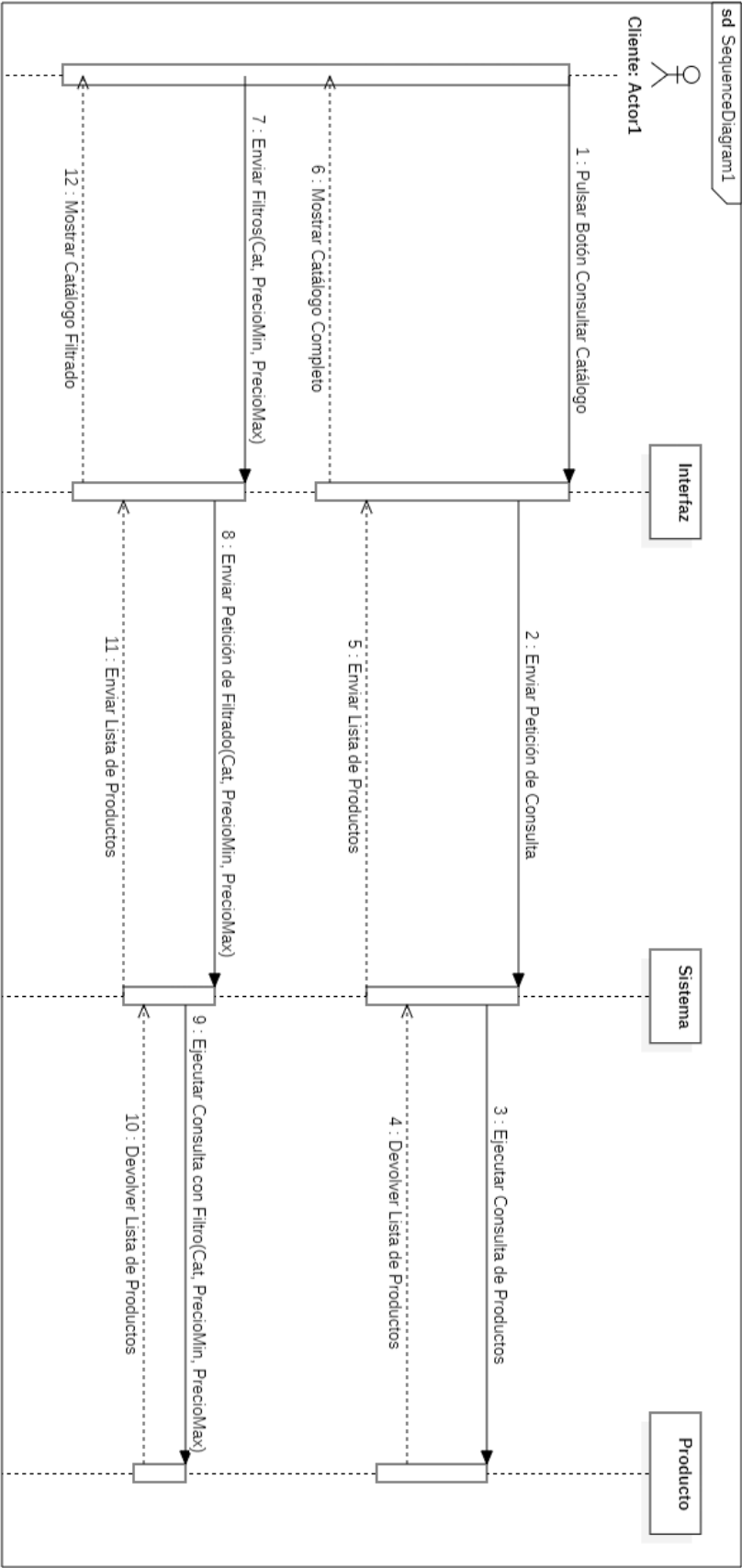
4.2.2 INSERTAR PAGO



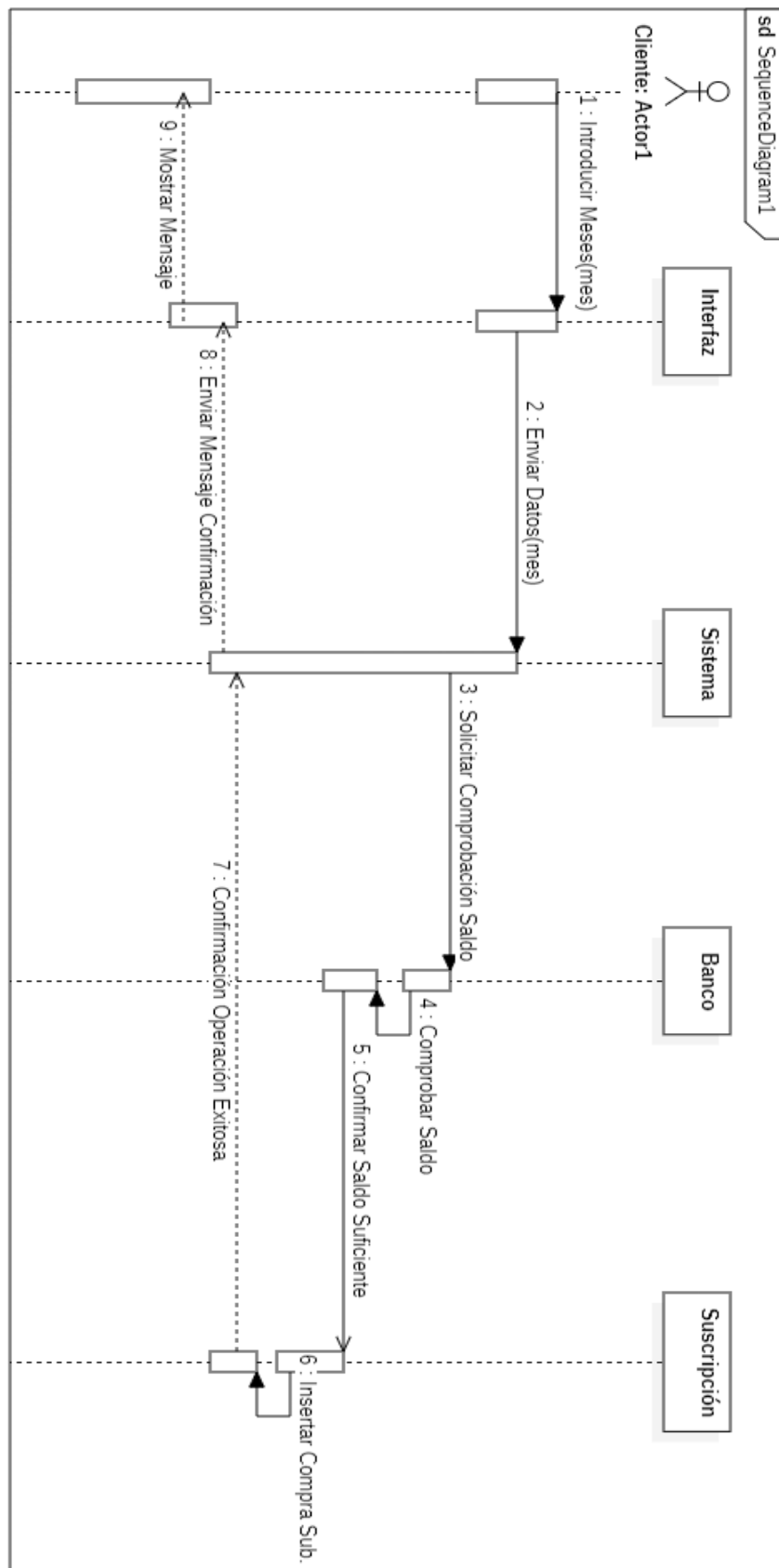
4.2.3 INSERTAR RESEÑA



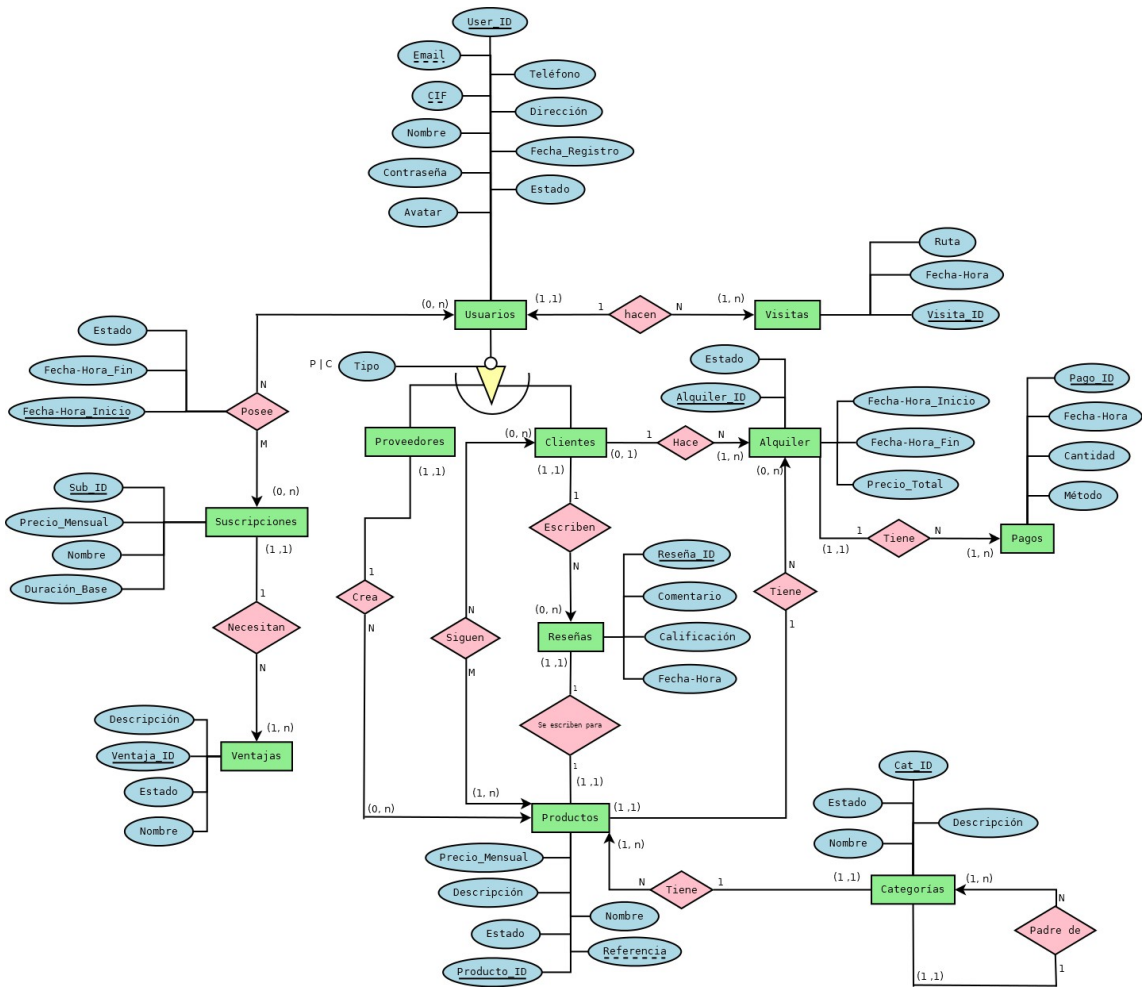
4.2.4 CONSULTAR CATÁLOGO DE PRODUCTOS



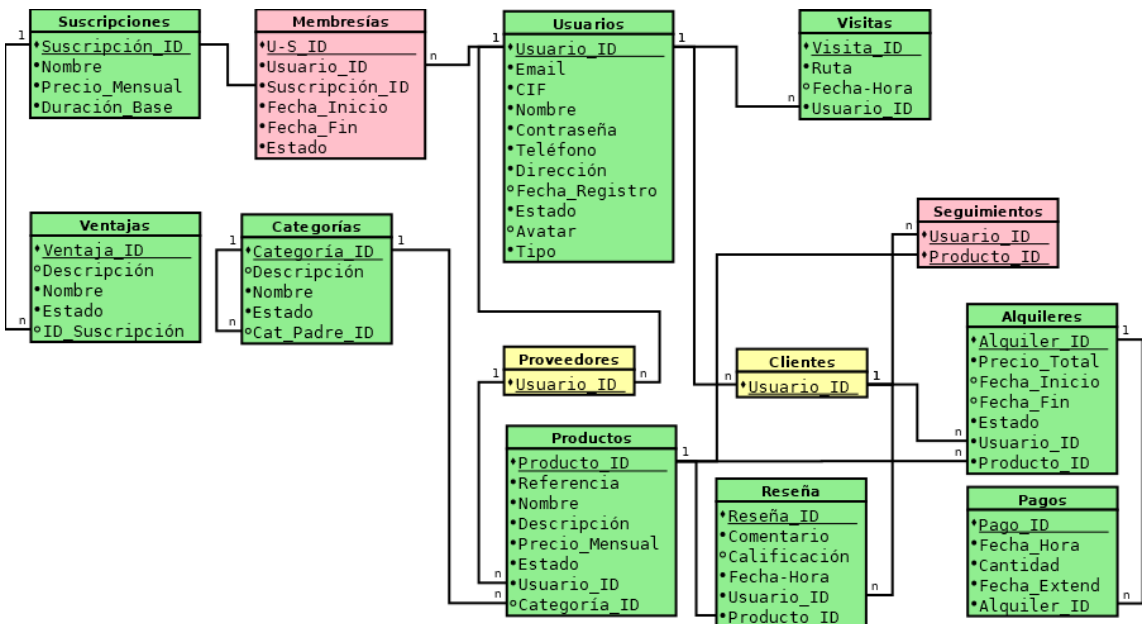
4.2.5 ADQUIRIR SUSCRIPCIÓN



4.3 DIAGRAMA MODELO ENTIDAD-RELACIÓN (MER)



4.4 DIAGRAMA TABLAS RELACIONALES



4.5 TABLAS DE LA BASE DE DATOS

4.5.1 USUARIOS

Información de los usuarios que se registran en la web.

Campo	Tipo	Descripción
USUARIO_ID	INT PRIMARY KEY	Identificador único del usuario
EMAIL	VARCHAR(100)	Correo electrónico único del usuario
CIF	VARCHAR(9)	Código CIF único de la empresa del usuario
NOMBRE	VARCHAR(100)	Nombre único de la empresa del usuario
CONTRASEÑA	VARCHAR(20)	Contraseña de la cuenta del usuario
DIRECCIÓN	VARCHAR(255)	Dirección física de la empresa del usuario
TELÉFONO	VARCHAR(9)	Teléfono único del usuario
TIPO	ENUM (P, C)	P para Proveedores, C para Clientes
AVATAR	BLOB	URL de la imagen de perfil del usuario
ESTADO	BOOL	1 = activo, 0 = inactivo
FECHA_REGISTRO	DATETIME	Fecha y hora en la que se registró el usuario

4.5.2 CLIENTES

Son los usuarios que alquilan productos y escriben reseñas sobre estos.

Campo	Tipo	Descripción
USUARIO_ID	INT PRIMARY KEY	Identificador del cliente asociado al campo USUARIO_ID

4.5.3 PROVEEDORES

Son los usuarios que suben sus propios productos a la web.

Campo	Tipo	Descripción
USUARIO_ID	INT PRIMARY KEY	Identificador del proveedor asociado al campo USUARIO_ID

4.5.4 VISITAS

Las páginas que visita cada usuario, para monitorear el flujo de navegación de la web.

Campo	Tipo	Descripción
VISITA_ID	INT PRIMARY KEY	Identificador de la visita a la web
RUTA	VARCHAR(255)	URL de la página visitada
FECHA_HORA	DATETIME	Fecha y Hora en la que se realizó la visita
USUARIO_ID	INT FOREIGN KEY	ID del usuario que entró en la página

4.5.5 SUSCRIPCIONES

Los usuarios pueden comprar una suscripción mensual y disfrutar de ciertas ventajas.

Campo	Tipo	Descripción
SUSCRIPCION_ID	INT PRIMARY KEY	Identificador de la suscripción
NOMBRE	VARCHAR(100)	Nombre de la suscripción
PRECIO_MENSUAL	DECIMAL(10, 2)	Precio de la suscripción mensualmente
DURACIÓN_BASE	INT	Meses que dura la suscripción al comprarla

4.5.6 VENTAJAS

Cada uno de los beneficios que otorgan las suscripciones.

Campo	Tipo	Descripción
VENTAJA_ID	INT PRIMARY KEY	Identificador de la ventaja
NOMBRE	VARCHAR(100)	Título de la ventaja
DESCRIPCIÓN	VARCHAR(255)	Descripción del beneficio que otorga la ventaja
SUSCRIPCION_ID	INT FOREIGN KEY	ID de la suscripción a la que pertenece

4.5.7 MEMBRESÍAS

Cada suscripción comprada por un usuario.

Campo	Tipo	Descripción
MEMBRESÍA_ID	INT PRIMARY KEY	Identificador de cada membresía
USUARIO_ID	INT FOREIGN KEY	ID del usuario que se ha suscrito.
SUSCRIPCION_ID	INT FOREIGN KEY	ID de la suscripción comprada por el usuario.
FECHA_INICIO	DATETIME	Fecha de compra.
FECHA_FIN	DATETIME	Fecha en la que se acabará la membresía
ESTADO	BOOL	1 = activa, 0 = terminada

4.5.8 CATEGORÍAS

Cada producto se puede clasificar según una categoría u otra.

Campo	Tipo	Descripción
CATEGORÍA_ID	INT PRIMARY KEY	Identificador de cada categoría
NOMBRE	VARCHAR(100)	Título de cada categoría
DESCRIPCIÓN	VARCHAR(255)	Descripción de cada categoría.
CAT_PADRE_ID	INT FOREIGN KEY	Cada categoría puede ser hija de otra.

4.5.8 PRODUCTOS

Cada proveedor sube productos a la web para que sean alquilados por los clientes

Campo	Tipo	Descripción
PRODUCTO_ID	INT PRIMARY KEY	Identificador del producto
NOMBRE	VARCHAR(100)	Nombre del producto
DESCRIPCIÓN	VARCHAR(255)	Descripción de las características del producto.
REFERENCIA	CHAR(5)	Un código formado por una letra mayúscula y 4 números como identificador único.
PRECIO_MENSUAL	DECIMAL(10, 2)	Coste del alquiler del producto mensualmente.
IMAGEN	VARCHAR(255)	URL de la imagen del producto
ACTIVO	BOOL	1 = activo, 0 = inactivo o alquilado
USUARIO_ID	INT FOREIGN KEY	Proveedor dueño del producto
CATEGORÍA_ID	INT FOREIGN KEY	Categoría a la que pertenece el producto

4.5.9 RESEÑAS

Cada cliente que ha alquilado el producto en cuestión al menos una vez puede dejar un comentario sobre el producto en la página de éste, y valorarlo de 0 a 5 estrellas.

Campo	Tipo	Descripción
RESEÑA_ID	INT PRIMARY KEY	Identificador de la reseña o comentario
COMENTARIO	VARCHAR(255)	Texto de la reseña
CALIFICACIÓN	INT	Un número del 0 al 5 que indica el grado de satisfacción del cliente con el producto
FECHA_HORA	DATETIME	Fecha y Hora en la que se escribió la reseña
USUARIO_ID	INT FOREIGN KEY	ID del usuario autor de la reseña
PRODUCTO_ID	INT FOREIGN KEY	ID del producto reseñado.

4.5.10 SEGUIMIENTOS

Cada usuario puede añadir o eliminar el producto que desee a una lista de “favoritos” o “seguimiento”, a modo de marcador para localizarlos fácilmente.

Campo	Tipo	Descripción
USUARIO_ID	INT FOREIGN KEY	ID del usuario interesado en el producto
PRODUCTO_ID	INT FOREIGN KEY	ID del producto añadido a seguimiento

4.5.11 ALQUILERES

Cada alquiler que un usuario realiza se almacena con los siguiente datos:

Campo	Tipo	Descripción
ALQUILER_ID	INT PRIMARY KEY	Identificador de cada alquiler exitoso
PRECIO_TOTAL	DECIMAL (10, 2)	Precio total del alquiler, se actualiza cada vez que el usuario decide renovarlo
FECHA_INICIO	DATETIME	Fecha en la que se realizó el alquiler
FECHA_FIN	DATETIME	Fecha de finalización del alquiler, se actualiza cada vez que el usuario decide renovarlo
ESTADO	BOOL	1 = activo, 0 = finalizado
USUARIO_ID	INT FOREIGN KEY	ID del cliente que realiza el alquiler
PRODUCTO_ID	INT FOREIGN KEY	ID del producto alquilado

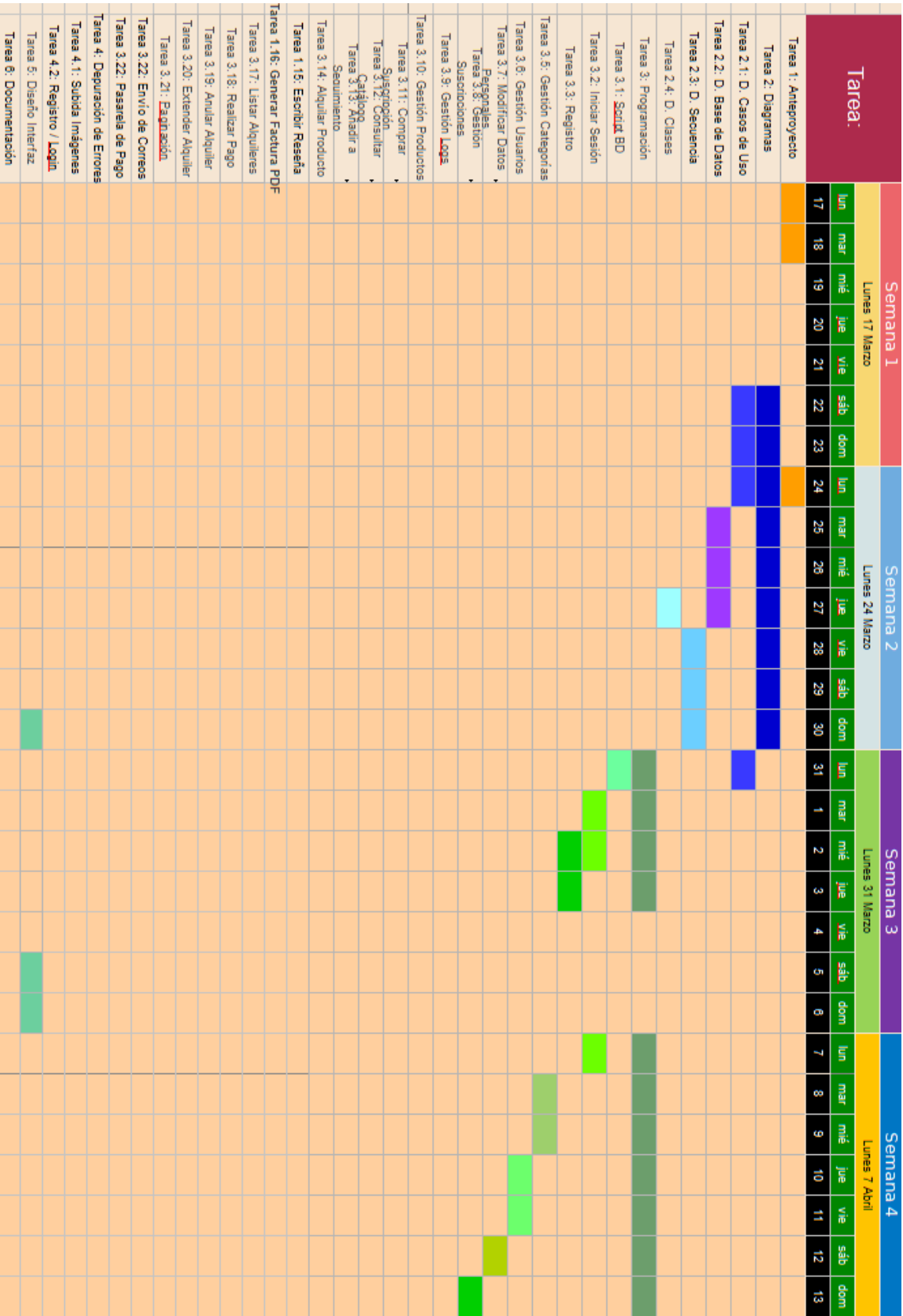
4.5.12 PAGOS

Cada vez que el usuario decide pagar el alquiler total o parcialmente y/o renovar el alquiler se almacena con los siguientes datos:

Campo	Tipo	Descripción
PAGO_ID	INT PRIMARY KEY	Identificador del pago realizado
CANTIDAD	INT	Dinero pagado en euros
MESES_EXTRA	INT	Meses que el usuario decide renovar el alquiler
FECHA_HORA	DATETIME	Fecha y Hora de realización del pago
ALQUILER_ID	INT FOREIGN KEY	ID del alquiler sobre el que se realiza el pago

4.5 DIAGRAMAS DE GANT

4.5.1 MARZO – ABRIL



4.5.2 ABRIL-MAYO

Tarea:	Semana 5							Semana 6							Semana 7							Semana 8						
	Lunes 14 de Abril							Lunes 21 de Abril							Lunes 28 de Abril							Lunes 5 de Mayo						
	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom
Tarea 1: Anteproyecto																												
Tarea 2: Diagramas																												
Tarea 2.1: D. Casos de Uso																												
Tarea 2.2: D. Base de Datos																												
Tarea 2.3: D. Secuencia																												
Tarea 2.4: D. Clases																												
Tarea 3: Programación																												
Tarea 3.1: Scrit BD																												
Tarea 3.2: Iniciar Sesión																												
Tarea 3.3: Registro																												
Tarea 3.5: Gestión Categorías																												
Tarea 3.6: Gestión Usuarios																												
Tarea 3.7: Modificar Datos																												
Personajes																												
Tarea 3.8: Gestión Suscripciones																												
Tarea 3.9: Gestión Logs																												
Tarea 3.10: Gestión Productos																												
Tarea 3.11: Comprar																												
Suscripción																												
Tarea 3.12: Consultar																												
Catálogo																												
Tarea 3.13: Añadir a Seguimiento																												
Tarea 3.14: Alquiler Producto																												
Tarea 1.15: Escribir Reseña																												
Tarea 1.16: Generar Factura PDF																												
Tarea 3.17: Listar Alquileres																												
Tarea 3.18: Realizar Pago																												
Tarea 3.19: Anular Alquiler																												
Tarea 3.20: Extender Alquiler																												
Tarea 3.21: Eliminación																												
Tarea 3.22: Envío de Correos																												
Tarea 3.22: Pasarela de Pago																												
Tarea 4: Depuración de Errores																												
Tarea 4.1: Subida Imágenes																												
Tarea 4.2: Registro / Login																												
Tarea 5: Diseño Interfaz																												
Tarea 6: Documentación																												

4.5.2 MAYO-JUNIO

Tarea:	Semana 9							Semana 10							Semana 11							Semana 12						
	Lunes 12 de Mayo							Lunes 19 de Mayo							Lunes 26 de Mayo							Lunes 2 de Junio						
	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom
Tarea 1: Anteproyecto																												
Tarea 2: Diagramas																												
Tarea 2.1: D. Casos de Uso																												
Tarea 2.2: D. Base de Datos																												
Tarea 2.3: D. Secuencia																												
Tarea 2.4: D. Clases																												
Tarea 3: Programación																												
Tarea 3.1: Script BD																												
Tarea 3.2: Iniciar Sesión																												
Tarea 3.3: Registro																												
Tarea 3.5: Gestión Categorías																												
Tarea 3.6: Gestión Usuarios																												
Tarea 3.7: Modificar Datos																												
Personajes																												
Tarea 3.8: Gestión Suscripciones																												
Tarea 3.9: Gestión Logs																												
Tarea 3.10: Gestión Productos																												
Tarea 3.11: Comprar																												
Suscripción																												
Tarea 3.12: Consultar																												
Catalogo																												
Tarea 3.13: Añadir a Seguimiento																												
Tarea 3.14: Alquiler Producto																												
Tarea 1.15: Escribir Reseña																												
Tarea 1.16: Generar Factura PDF																												
Tarea 3.17: Listar Alquileres																												
Tarea 3.18: Realizar Pago																												
Tarea 3.19: Anular Alquiler																												
Tarea 3.20: Extender Alquiler																												
Tarea 3.21: Evaluación																												
Tarea 3.22: Envío de Correos																												
Tarea 3.22: Pasarela de Pago																												
Tarea 4: Depuración de Errores																												
Tarea 4.1: Subida Imágenes																												
Tarea 4.2: Registro / Login																												
Tarea 5: Diseño Interfaz																												
Tarea 6: Documentación																												

5. GUÍA DE ESTILOS

5.1 TIPOGRAFÍA

Para esta aplicación se utilizan dos tipos de fuente de letra principales:

- **OPEN SANS:** Para textos en el cuerpo de la aplicación.

Whereas disregard and contempt for
human rights have resulted

- **MARCELLUS:** Para botones de navegación y títulos.

Whereas disregard and contempt for
human rights have resulted

5.2 COLORES

Para esta web se utilizan los siguientes colores colores principales:

#116530 Color Principal	#2d465e Encabezados	
#fff8dc, #f4e2d8, #deb887 Cards de producto	#FFC1E3, #D6A4EC Cards de suscripciones	
Limegreen	Orange	Crimson
Extender Suscripcion Ver y Crear Ventajas	Crear Pago Modificar Datos	Anular Suscripcion Eliminar Datos

5.3 ICONOS

Se han utilizado algunos iconos gratis ofrecidos por la web FontAwesome, por ejemplo:

- **FA-GEAR** y **FA-TRASH:** Modificar y Eliminar Datos.
- **FA-PLUS:** Extender Alquiler, Añadir Ventajas a una suscripción (Admin).
- **FA-STAR:** Icono en el que pulsar para marcar o desmarcar productos favoritos.
- **FA-WHEAT-AWN:** Gestionar Productos.
- **FA-CREDIT-CARD:** Realizar Pago.
- **FA-RIGHT-TO-BRACKET:** Iniciar y Cerrar Sesión.

5.4 IMÁGENES

Logo de la aplicación:



Fondo principal de las secciones de funcionalidad de la web:



6. DESARROLLO DE LA APLICACIÓN

6.1 PROGRAMACIÓN DEL LADO SERVIDOR

6.1.1 MODELO VISTA CONTROLADOR (MVC)

Para el desarrollo óptimo y organizado de esta aplicación, se ha creado un controlador y un modelo para cada una de las tablas existentes de la base de datos, menos para Clientes y Proveedores, cuyas funciones se pueden incluir en el controlador y modelo de usuario al tener un único campo (USUARIO_ID).

Además, cada tabla también posee un “Índice” que es un documento que se incluirá en la página principal de la web al navegar por ella, que contiene las llamadas a las diferentes funciones que puede realizar cada Controlador.

En general, las funciones esenciales que contiene cada Controlador y Modelo son las siguientes, que corresponden a las funciones CRUD de una Base de Datos:

- **SELECCIÓN COMPLETA:** Seleccionar todas las filas existentes en la tabla, en muchos de ellos utilizando un parámetro llamado “\$offset” utilizado para la paginación del listado de la información recogida.
- **SELECCIÓN CONCRETA:** Seleccionar los datos de una única fila, para lo cual se utilizan los parámetros “\$field” y “\$value” (campo mediante el que buscar y valor de este campo).
- **INSERCIÓN:** Añadir los datos a su tabla correspondiente de la base de datos. En algunos casos, los datos se envían al modelo mediante una cookie codificada que los contiene, mientras que en otros, los datos se envían como parámetros directamente.
- **ACTUALIZACIÓN:** Modificar los datos de una fila en específico. Al igual que con las operaciones de Inserción, algunas se realizan enviando los datos al Modelo como una cookie codificada, pero en todos ellos se envía como parámetro la id del objeto a actualizar. No existe para algunos Modelos, como el correspondiente a la tabla “Pagos”, al no ser necesario.
- **ELIMINACIÓN:** Eliminar la fila seleccionada de la base de datos. Se pasa como parámetro el ID del objeto.
- **ACTIVACIÓN:** Para aquellas tablas que tienen un campo booleano “Estado”, sus modelos poseen un método que cambia el estado del objeto en cuestión (pasando su ID como parámetro).


```

/* Función: Insertar un nuevo alquiler junto al primer pago y desactivar el producto alquilado
 * Params: $pId (ID del producto), $month (meses de alquiler), $price (precio del alquiler)
 * Return: Mensaje de éxito o error
 */
public function insertRent(&$pId, &$month, &$price){
    if(isset($pId) && isset($month) && isset($price) && $pId!="" && $month!="" && $price!=""){
        include("_Indexes/Index_Product.php");
        $field="Producto_ID"; $active=0;

        if($productController -> selectProduct($field, $pId)[0]["Estado"]==0)
            header("Location: principal.php?methodProd=viewProduct&id=".$pId."&page=1&success=1");
        else{
            if($this->rentModel->insertRent($pId, $month, $price)==1){
                $rentControl=$this->selectLastRent();
                $productController -> activeProduct($pId, $active);

                include("_Indexes/Index_Pay.php");
                if($payController->insertPay($rentControl[0]["Alquiler_ID"], $price, $month, $price)==1){
                    setcookie("data-rent", 0, time() - 3600, "/");
                    include("Views/Client_Rent_Ok.php");
                }
            }else
                return "Error al alquilar producto.";
        }
    }else
        return "Error inesperado";
}
}

/* Función: Insertar un nuevo alquiler
 * Params: $id (ID del producto), $month (meses de alquiler), $price (precio del alquiler)
 * Return: 1 o -1 en caso de error
 */
public function insertRent(&$id, &$month, &$price){
    try{
        $sql=$this->db->prepare("INSERT INTO ALQUILERES (USUARIO_ID, PRODUCTO_ID, FECHA_INICIO, FECHA_FIN, ESTADO, PRECIO_TOTAL)
        VALUES ((SELECT USUARIO_ID FROM USUARIOS WHERE Nombre=?), ?, ?, ?, ?, ?)");
        $sql->bindValue(1, $_SESSION["usuario"], PDO::PARAM_STR);
        $sql->bindValue(2, $id, PDO::PARAM_INT);
        $sql->bindValue(3, date("Y-m-d H:i:s"), PDO::PARAM_STR);
        $sql->bindValue(4, date("Y-m-d H:i:s", strtotime("+".$month." months")), PDO::PARAM_STR);
        $sql->bindValue(5, 1, PDO::PARAM_INT);
        $sql->bindValue(6, $price*$month, PDO::PARAM_INT);
        $sql->execute();

        return 1;
    }catch(PDOException $e) {
        return -1;
    }
}
}

```

Ejemplo de la función de inserción en el Controlador y Modelo de Alquileres


```

/* Función: Actualizar un producto
 * Params: $id (ID del producto)
 * Return: Redirección a la página principal con éxito o error, o mensaje de éxito/error
 */
public function updateProduct(&$id){
    if(isset($_COOKIE["data-prod"])){
        $data = json_decode($_COOKIE["data-prod"], true);
        if($this->productModel->updateProduct($id, $data)==1){
            setcookie("data-prod", 0, time()-1,"/");
            setcookie("product-image", 0, time()-1,"/");
            header("Location: principal.php?methodProd=select&action=update");
        }
    }else{
        return "No se han recibido datos para actualizar el producto.";
    }
}

```

```

/* Función: Actualizar un producto
 * Params: $id (ID del producto), $data (datos del producto)
 * Return: Redirección a la página principal con éxito o error, o mensaje de éxito/error
 */
public function updateProduct(&$id, &$data){
    try{
        if(isset($_COOKIE["product-image"])) $file=$_COOKIE["product-image"];
        else{
            $field="Producto_ID";
            $prod=$this->selectProduct($field, $id)[0]['Imagen'];
            if(is_array($prod)) $file=$prod[0]['Avatar'];
            else $file=null;
        }

        $sql=$this->db->prepare("UPDATE PRODUCTOS SET Nombre=?, Descripción=?, Referencia=?,
                                Precio_Mensual=?, Categoría_ID=?, Imagen=? WHERE Producto_ID=?");
        $sql->bindValue(1, $data[0]);
        $sql->bindValue(2, $data[1]);
        $sql->bindValue(3, $data[2]);
        $sql->bindValue(4, $data[3]);
        if (empty($data[4])) $sql->bindValue(5, null, PDO::PARAM_NULL);
        else $sql->bindValue(5, $data[4], PDO::PARAM_INT);
        $sql->bindValue(6, $file);
        $sql->bindValue(7, $id, PDO::PARAM_INT);
        $sql->execute();

        return 1;
    }catch(PDOException $e) {
        return -1;
    }
}

```

Ejemplo de la función de actualización en el Controlador y Modelo de Producto.
 Nótese que “\$data” es un array de datos que se crea al validar el formulario mediante
 la función de Javascript [encodeURIComponent\(JSON.stringify\(\)\)](#)

```

/* Función: Procesar la imagen del producto
 * Params: $id (ID del producto), $file (archivo de imagen)
 * Return: Nombre de la imagen procesada
 */
public function imageProcess(&$id, &$file){
    if($id!=null && $id!=""){
        $field="Producto_ID";
        $oldImage = $this->selectProduct($field, $id)[0]['Imagen'];
        if($oldImage) unlink("../assets/img/products/$oldImage");
        $extension = pathinfo($file['name'], PATHINFO_EXTENSION);
        $image = $id . "." . $extension;
    }else{
        $idName = $this->db->prepare("SELECT MAX(PRODUCTO_ID) AS 'LastID' FROM PRODUCTOS");
        $idName->execute();
        $idName = $idName->fetch(PDO::FETCH_ASSOC)['LastID'];
        if($idName==null) $idName=1; else $idName++;
        $extension = pathinfo($file['name'], PATHINFO_EXTENSION);
        $image = $idName . "." . $extension;
    }

    $imagentemp = $file["tmp_name"];
    move_uploaded_file($imagentemp, "../assets/img/products/".$image);
    setcookie("product-image", $image, time()+3600, "/");
    return $image;
}

```

Función de procesamiento de imagen del Modelo de Producto, encargada de renombrar y subir la imagen al servidor, creando una cookie con su ruta

Otros métodos que merece la pena mencionar son dos métodos incluidos en los Controladores de Alquileres y Membresías, que comprueban si la fecha y hora actual es mayor que la fecha de fin, y en caso afirmativo, desactivan las filas afectadas y muestra un mensaje emergente durante unos segundos en la parte superior de la pantalla.

```

/* Función: Comprobar si hay alquileres que deberían desactivarse y notificar al usuario
 * Params: $offset (paginación) seteado por referencia a -1 lo cual selecciona todos los alquileres del usuario
 * Return: Notificación de alquiler activo si corresponde
 */
public function checkActiveRent(&$offset){
    if(isset($_SESSION["usuario"]) && $_SESSION["usuario"]!="ADMINISTRADOR"){
        $rentControl = $this->rentModel->listRent($offset);

        if(is_array($rentControl)){
            foreach($rentControl as $rent){
                $timeDiffFin=strtotime($rent["Fecha_Fin"]) - strtotime(date("Y-m-d H:i:s"));

                if($timeDiffFin<=0){
                    if($this->activeRent($rent["Alquiler_ID"], $rent["PID"])==1){
                        ?> <script>showBoxActiveRent( "<?php echo $rent["Nombre"]; ?>" );</script> <?php
                    }
                }
            }
        }
    }
}

```

Método de desactivación automática de Alquileres del Usuario

```

/* Función: Desactivar un alquiler
 * Params: $rId (ID del alquiler)
 * Return: Mensaje de éxito o error
 */
public function activeRent(&$rId, &$pId){
    $rentControl = $this->rentModel->activeRent($rId);
    if($rentControl==1){
        include("_Indexes/Index_Product.php"); $active=1;
        $productController -> activeProduct($pId, $active);

        return 1;
    }else
        return "Error inesperado";
}

```

Nótese que, si el alquiler se desactiva con éxito, se llama al controlador y al modelo de producto para ejecutar el método de activación del producto.

Como se ve en la imagen superior, la ejecución exitosa de la desactivación de un alquiler provoca la invocación de otro MVC y la llamada a uno de sus métodos. Esto es algo que también se da al insertar un Alquiler o al insertar un Pago:

```

/* Función: Insertar un nuevo alquiler junto al primer pago y desactivar el producto alquilado
 * Params: $pId (ID del producto), $month (meses de alquiler), $price (precio del alquiler)
 * Return: Mensaje de éxito o error
 */
public function insertRent(&$pId, &$month, &$price){
    if(isset($pId) && isset($month) && isset($price) && $pId!="" && $month!="" && $price!=""){
        include("_Indexes/Index_Product.php");
        $field="Producto_ID"; $active=0;

        if($productController -> selectProduct($field, $pId)[0]["Estado"]==0)
            header("Location: principal.php?methodProd=viewProduct&id=".$pId."&page=1&success=0");
        else{
            if($this->rentModel->insertRent($pId, $month, $price)==1){
                $rentControl=$this->selectLastRent();
                $productController -> activeProduct($pId, $active);

                include("_Indexes/Index_Pay.php");
                if($payController->insertPay($rentControl[0]["Alquiler_ID"], $price, $month, $price)==1){
                    setcookie("data-rent", 0, time() - 3600, "/");
                    include("Views/Client_Rent_Ok.php");
                }
            }else
                return "Error al alquilar producto.";
        }
    }else
        return "Error inesperado";
}

```

*Comprobar Estado del Producto > Desactivar Producto >
Insertar Primer Pago (equivalente al primer mes de alquiler)*

```

/* Función: Insertar pago
 * Params: $rId (ID del alquiler), $mPrice (precio del mes),
 * $month (mes a pagar), $amount (cantidad a pagar, por defecto 0)
 * Return: Redirección a la página principal con éxito o error, o mensaje de éxito/error
 */
public function insertPay(&$rId, &$mPrice, &$month, &$amount=0){
    if($this->payModel->insertPay($rId, $month, $amount)==1){
        if($month!="" && !isset($_COOKIE["data-rent"])){
            include("_Indexes/Index_Rent.php");
            $rentController->updateRent($month, $mPrice, $rId);
        }

        if(isset($_GET["methodRent"])) return 1;
        else if(isset($_GET["methodPay"])){
            setcookie("data-pay", "", time() - 3600, "/");
            header("Location: principal.php?methodRent=select&page=1&success=1");
        }else
            return "Pago realizado correctamente";
    }else{
        if(isset($_GET["methodPay"]))
            header("Location: principal.php?methodRent=select&page=1&success=0");
        else
            return "Error al realizar el pago";
    }
}
}

```

Insertar Pago. Si no es el primer pago, el cual se inserta automáticamente, se llama a la función de actualización del alquiler en caso de que el usuario desee extenderlo.

Nótese las cookies, las cuales se crean durante el pago mediante Stripe

Llamada al método de actualización del alquiler si el usuario desea extender su duración al realizar un pago.

6.1.2 STRIPE - PASARELA DE PAGO

Stripe es una plataforma que permite implementar funciones de pago electrónico de manera segura y eficiente. Una característica destacada de esta plataforma es su capacidad para simular pagos mediante su entorno de pruebas (sandbox). Esto permite a desarrolladores probar flujos completos de pago sin usar dinero real. Usando tarjetas de prueba y escenarios simulados, pueden verificar cómo responde el sistema ante transacciones exitosas o fallidas.

Para implementar esta pasarela de pago, en primer lugar se ha utilizado el siguiente comando para descargar la API oficial de Stripe dentro del proyecto:

composer require stripe/stripe-php

```
require '../assets/vendor/Stripe/vendor/autoload.php';
\Stripe\Stripe::setApiKey('secret_key_here');
```

Método de carga de la API de Stripe.

Nuestra clave secreta la obtendremos al crear una cuenta en la web oficial

```
/* Función: Crea una sesión de Stripe Checkout y redirige al usuario
 * Params: price (precio del producto)
 * Return: Redirección a Stripe Checkout
 */
try {
    $session = \Stripe\Checkout\Session::create([
        'payment_method_types' => ['card'],
        'line_items' => [[
            'price_data' => [
                'currency' => 'eur',
                'product_data' => ['name' => 'Operación de pago'],
                'unit_amount' => intval($price*100),
            ],
            'quantity' => 1,
        ]],
        'mode' => 'payment',
        'success_url' => $linkSuccess,
        'cancel_url' => 'http://localhost/FARMXPRESS/index.php',
    ]);

    // Redirect to Stripe Checkout
    header("Location: " . $session->url);
    exit();
} catch (Exception $e) {
    header("Location: http://localhost/FARMXPRESS/principal.php?methodProd=select&page=1&error=1");
}
```

Creación de sesión para la realización de pago

```

$linkSuccess="https://localhost/FARMXPRESS/include/";

/* Función: Recoge datos de pago y crea una sesión de Stripe Checkout
 * Params: pId (ID del producto), price (precio del producto), month (meses de alquiler)
 * Return: Cookie con los datos del alquiler o pago, redirige a Stripe Checkout
 */
if(isset($_GET['methodRent'])){
    $price= isset($_POST['price']) ? $_POST['price'] : 0;
    $cookie_array=setcookie("data-rent", json_encode([
        'pId' => isset($_POST['pId']) ? $_POST['pId'] : 0,
        'month' => isset($_POST['month']) ? $_POST['month'] : 0,
        'price' => $price
    ]), time() + (86400 * 30), "/");

    $linkSuccess.="principal.php?methodRent=insert";
}

}else if(isset($_GET['methodPay'])){
    $price= isset($_POST['amount']) ? $_POST['amount'] : 0;
    $cookie_array=setcookie("data-pay", json_encode([
        'rId' => isset($_POST['rId']) ? $_POST['rId'] : 0,
        'mPrice' => isset($_POST['mPrice']) ? $_POST['mPrice'] : 0,
        'month' => isset($_POST['month']) ? $_POST['month'] : 0,
        'amount' => $price,
    ]), time() + (86400 * 30), "/");

    $linkSuccess.="principal.php?methodPay=insert";
}

}else if(isset($_GET['methodMember'])){
    $data = json_decode($_COOKIE['data-member'], true);
    $price= isset($data['price']) ? $data['price'] : 0;

    $linkSuccess.="principal.php?methodMember=insert";
}

```

Manera en la que se manipulan los datos. Las cookies “data-rent” y “data-pay”, vistas anteriormente, se crean junto a la sesión de pago de Stripe Checkout.

← FarmXPRESS sandbox Sandbox

Operación de pago
€200.00

Pay with card

Email
email@example.com

Card information
1234 1234 1234 1234 VISA M C A
MM / YY CVC

Cardholder name
Full name on card

Country or region
Spain

☐ Securely save my information for 1-click checkout
Pay faster on FarmXPRESS sandbox and everywhere Link is accepted.

Pay

Ejemplo de visualización de Stripe Checkout para el alquiler de un producto que cuesta 200€ mensualmente

6.2.1 VALIDACIÓN DE DATOS DE FORMULARIO DE REGISTRO

Para ello, se empieza seleccionando los valores de todos los campos de formulario, se recortan espacios con la función trim() y se declaran las expresiones regulares necesarias para el correcto formateo de los campos.

Ejemplo de validación del campo Email para el formulario de registro de usuario o de modificación de sus datos

Para ello se utiliza la siguiente función asíncrona, `validateField`. Sus tres parámetros consisten en:

- 31

```

async function validateField(field, value, fieldName) {
    if($('#userId').length > 0) var id = $('#userId').val().trim();
    else var id= null;

    return new Promise((resolve, reject) => {
        $.ajax({
            url: '../assets/js/validateUser.php',
            type: 'POST',
            dataType: 'json',
            data: { field: field, value: value, userId: id },
            success: function(response) {
                if(response.text != 0){
                    $('#error').text("Ya hay un usuario registrado con este " + field);
                    $('#'+ fieldName).val("");
                    $('#'+ fieldName).focus();
                    resolve(false);
                } else {
                    $('#error').text("");
                    resolve(true);
                }
            },
            error: function(xhr, status, error) {
                $('#error').text("Error inesperado");
                reject(error);
            }
        });
    });
}

```

Esta función devolverá una Promesa, que se utiliza para marcar su llamada como “await”, es decir, que el programa tenga que esperar a su ejecución

Por cuestiones técnicas, la función de validación se ha debido guardar en un documento aparte, en lugar de ser incluida en el MVC de Usuarios. El campo “#userId” es un formulario oculto que solo aparece al acceder a la sección de modificar nuestros datos (o de cualquier usuario en caso del Administrador).

```

<?php
$c = new PDO("mysql:host=localhost;dbname=FARMXPRESS", "root", "");
$c->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
header('Content-Type: application/json');

if($_POST['userId']!=null)
    $query="SELECT * FROM USUARIOS WHERE " . $_POST["field"] . " = :value AND USUARIO_ID != :userId";
else
    $query="SELECT * FROM USUARIOS WHERE " . $_POST["field"] . " = :value";

$sql= $c->prepare($query);
$sql->bindParam(':value', $_POST["value"]);
if($_POST['userId']!=null)
    $sql->bindParam(':userId', $_POST["userId"]);

$sql->execute();
$result = $sql->fetchAll(PDO::FETCH_ASSOC);

if (is_array($result) && count($result) > 0)
    $result = $result[0][$_POST["field"]];
else $result = 0;

echo json_encode(["text" => $result]);
exit;
?>

```

Código de validación. Si existe una ID, se ignora la fila con esa ID, para no afectar a los campos que el usuario no desee modificar

Como ya se ha dicho, este mismo formulario se utiliza tanto para registros como modificar datos del usuario. La modificación no permite modificar la contraseña, por lo que el valor se inicializa vacío en el script y, si existe un campo cuyo atributo id es “#password”, el script toma su valor y realiza la validación correspondiente.

Por último, para la subida del avatar del usuario se realiza mediante el uso de una función AJAX que, tras comprobar la extensión y el tamaño de las imágenes, utiliza la clase FormData para enviar al Modelo los datos de la imagen y, en caso de querer cambiar la imagen de perfil, la ID del usuario para renombrar la imagen.

```
// Subir Avatar
if(file != "") {
    var fileExt = $("#avatar").val().split('.').pop().toLowerCase();
    if($.inArray(fileExt, ['png', 'jpg', 'jpeg']) == -1) {
        $("#error").text("El campo Avatar debe estar en formatos PNG, JPG o JPEG.");
        $("#avatar").focus();
        return;
    } else if ($("#avatar").get(0).files[0].size > 1000000) {
        $("#error").text("El campo Avatar no puede superar el 1MB.");
        $("#avatar").focus();
        return;
    } else {
        var formData = new FormData();
        var file=$("#avatar").get(0).files[0];
        if($("#userId").length > 0) var id = $("#userId").val().trim(); else var id="";

        formData.append('userId', id);
        formData.append('avatar', file);

        $.ajax({
            url: 'principal.php?methodUser=uploadAvatar',
            type: 'POST',
            data: formData,
            processData: false,
            contentType: false,
            success: function(response) {
                console.log('Upload response:', response.text);
            }
        });

        await sleep(500);
    }
}
async function sleep(ms) { return new Promise(resolve => setTimeout(resolve, ms)); }
```

La imagen de avatar se procesa en una función similar a la mostrada en la página 21, por supuesto, utilizando la tabla de Usuarios en lugar de Productos

Para finalizar, todos los datos se almacenan en una cookie (*data-user*) que se envía al MVC de la tabla Usuarios. Estos son los requisitos de validación:

Campo	Requisitos
EMAIL	Máximo 100 caracteres, debe contener varios caracteres, un @ seguido de varios caracteres, un punto y 2-3 caracteres (ej: .com)
CIF	Una letra mayúscula y 8 dígitos
NOMBRE	Máximo 100 caracteres
CONTRASEÑA	Máximo 20 caracteres
TELÉFONO	9 dígitos
DIRECCIÓN	Máximo 255 caracteres
AVATAR	Formato .jpg o .png y máximo 1MB de tamaño

6.2.2 VALIDACIÓN DE DATOS DE FORMULARIO DE PRODUCTO

Campo	Requisitos
NOMBRE	Máximo 100 caracteres
DESCRIPCIÓN	Máximo 255 caracteres
REFERENCIA	Una letra mayúscula y 4 dígitos
PRECIO MENSUAL	Un número positivo
IMAGEN	Formato .jpg o .png y máximo 1MB de tamaño

La imagen se valida y procesa de la misma forma que el Avatar de un usuario. Sólo el campo Referencia es único, el cual también se comprueba de la misma forma, enviando el dato a un documento aparte que comprueba si la Referencia ya existe en la Base de Datos (ignorando la fila con la ID del producto en caso de modificar sus datos).

6.2.3 FORMULARIO DE PAGO

Para evitar que el usuario realizar un pago introduciendo una cantidad negativa, o una cantidad superior a la cantidad pendiente, se selecciona la suma total de los pagos hechos sobre ese alquiler mediante el siguiente método del Modelo de la entidad Pagos:

```

/* Función: Recuperar deudas de un alquiler
 * Params: $rId (ID del alquiler)
 * Return: Cantidad total adeudada, 0 si no hay pagos, -1 en caso de error
 */
public function selectDebtMoney(&$rId){
    try{
        $sql=$this->db->prepare("SELECT SUM(CANTIDAD) AS 'Debt_Money' FROM PAGOS WHERE ALQUILER_ID=?");
        $sql->bindValue(1, $rId, PDO::PARAM_INT);
        $sql->execute();

        if($sql->rowCount() != 0)
            return $sql->fetch(PDO::FETCH_ASSOC);
        else
            return 0;
    } catch(PDOException $e) {
        return -1;
    }
}

```

Tras recabar la cantidad, se resta al precio total del Alquiler y se inserta en un formulario oculto, que servirá para recabar ese valor mediante JQuery y realizar las comprobaciones adecuadas.

```
$("#amount").on("input", function() {  
    var value = $(this).val();  
    var maxAmount = parseInt($("#maxAmount").val());  
    if (value > maxAmount) $(this).val(maxAmount);  
    else if (value < 1 || isNaN(value)) $(this).val(1);  
});
```

Si la cantidad es mayor que la cantidad debida, el valor del formulario pasa a ser esa cantidad. Si el valor del formulario no es un número o es uno negativo, se vuelve 1

6.2.4 SEGUIMIENTO DE PRODUCTOS

```
$(document).ready(function(){  
    for(let $i=0;$i<$(".card").length;$i++){  
        /*-----FAVORITE-----*/  
        $(".card").eq($i).find("a").eq(0).on("click", function(event){  
            event.preventDefault();  
  
            let id=$(this).attr("id").split("-")[1];  
            let action=$(this).attr("id").split("-")[0];  
  
            $.ajax({  
                url: "principal.php?methodFav=toggleFav",  
                type: "POST",  
                data: {prodId:id, action:action},  
                success: function(data){  
                    if(action=="add"){  
                        showBoxActiveFav(1);  
                        $(".card").eq($i).find("a").eq(0).attr("id", "del-"+id);  
                    }  
                    else if (action=="del"){  
                        showBoxActiveFav(0);  
                        $(".card").eq($i).find("a").eq(0).attr("id", "add-"+id);  
                    }  
                },  
                error: function(){  
                    alert("Error inesperado");  
                }  
            });  
            $(this).children("i").eq(0).toggleClass("fa-regular");  
            $(this).children("i").eq(0).toggleClass("fas");  
        });  
        /*-----FAVORITE-----*/  
    }  
});
```

El script superior, presente en la página del catálogo de productos, asigna esta función AJAX al primer elemento “a” de cada card de producto. El atributo id de cada uno se compone de dos partes separadas por un guión, la “acción” (add/añadir, y del/eliminar), y la ID del producto en cuestión. La operación envía tanto la acción como la ID al Modelo de “Favoritos”, y realiza una consulta de inserción o eliminación en la tabla de Favoritos gracias al ID enviado.

```

/* Función: Insertar Producto en Favoritos o eliminarlo
 * Params: $id (ID del producto), $action (add para añadir, del para eliminar)
 * Return: 1 si la operación fue exitosa, -1 en caso de error
 */
public function toggleFav(&$id, &$action){
    try{
        if($action=="add")
            $query="INSERT INTO SEGUIMIENTOS (Producto_ID, Usuario_ID) VALUES (?, (SELECT Usuario_ID FROM USUARIOS WHERE Nombre=?))";
        else if($action=="del")
            $query="DELETE FROM SEGUIMIENTOS WHERE Producto_ID=? AND Usuario_ID=(SELECT Usuario_ID FROM USUARIOS WHERE Nombre=?)";

        $sql=$this->db->prepare($query);
        $sql->bindValue(1, $id, PDO::PARAM_INT);
        $sql->bindValue(2, $_SESSION["usuario"], PDO::PARAM_STR);
        $sql->execute();
        return 1;
    }catch(PDOException $e) {
        return -1;
    }
}

```

La acción determina el tipo de consulta. Por eficiencia, el ID del usuario se selecciona mediante una subconsulta

En último lugar, todos tus productos añadidos a favoritos aparecerán al pulsar “Productos Favoritos” en la barra de navegación. Esta sección también contiene este script, para poder desmarcar los productos fácilmente.

6.2.5 GENERACIÓN DE PDFs

Al alquilar un producto, es posible generar una factura descargable en formato PDF con los datos del alquiler. La creación del PDF se realiza mediante la librería “PDF24”.

```

async function pdfCreate(){
    const rentDetails = document.getElementById("rent-details");
    const date = new Date();

    var archivoPDF = new PDF24Doc();
    archivoPDF.setCharset("UTF-8");
    archivoPDF.setFilename("FacturaFarmXpress.pdf");
    archivoPDF.setPageSize(210, 297);

    var contenidoPDF = new PDF24Element();
    contenidoPDF.setTitle("Factura de Alquiler");
    contenidoPDF.setUrl("http://www.pdf24.org");
    contenidoPDF.setAuthor("FarmXpress");
    contenidoPDF.setDateTime(date.getTime());
    var texto=rentDetails.innerHTML
    contenidoPDF.setBody(texto);
    archivoPDF.addElement(contenidoPDF);

    archivoPDF.create();
}

```

```

PDF24Doc.prototype.setPageSize = function(width, height) {
    this.setParam('pageSize', width + 'x' + height);
};
PDF24Doc.prototype.setEmailTo = function(emailAddr) {
    this.setParam('emailTo', emailAddr);
};
PDF24Doc.prototype.addEmailTo = function(emailAddr) {
    if(this.params.emailTo && this.params.emailTo != '') {
        this.params.emailTo += ';' + emailAddr;
    } else {
        this.params.emailTo = emailAddr;
    }
};
PDF24Doc.prototype.setEmailFrom = function(emailAddr) {
    this.setParam('emailFrom', emailAddr);
};
PDF24Doc.prototype.setEmailSubject = function(subject) {
    this.setParam('emailSubject', subject);
};
PDF24Doc.prototype.setEmailBodyType = function(bodyType) {
    this.setParam('emailBodyType', bodyType);
};
PDF24Doc.prototype.setEmailBody = function(body) {
    this.setParam('emailBody', body);
};
PDF24Doc.prototype.setEmailCharset = function(charset) {
    this.setParam('emailCharset', charset);
};

function PDF24Element() {
    this.params = new Object();
}
PDF24Element.prototype.setParam = function(paramKey, paramValue) {
    this.params[paramKey] = paramValue;
};
PDF24Element.prototype.setTitle = function(title) {
    this.setParam('title', title)

```

Extracto de la mencionada librería, con funciones para establecer ciertos atributos y características al archivo generado, con capacidad de trabajar con e-mails

6.2.5 CREACIÓN DE MENSAJES DE AVISO POPUP

Muchas de las acciones que se pueden realizar en esta web – inicio de sesión, realizar un pago, marcar un producto como favorito... se notifican al usuario como un mensaje de éxito o error que aparece momentáneamente en la parte superior de la pantalla al realizarlas.

Todos los mensajes están almacenados en un fichero Javascript llamado “popup_box_create.js” El funcionamiento general se basa en la redirección por parte de los controladores a las secciones de la web donde se realizan las acciones invocadas, pero añadiendo un nuevo parámetro a la queryString de la URL con valores como 1, 2, 0, -1... que determinan el mensaje que muestra el aviso.

```

function showBoxSuccessLogin(bool, name){
    const warning = document.createElement("div");

    if(bool==0){
        warning.textContent="Sesión cerrada correctamente.";
        warning.setAttribute("class", "warning alert alert-danger");
    }else if(bool==1){
        warning.textContent="Bienvenido " + name + ", has iniciado sesión correctamente";
        warning.setAttribute("class", "warning alert alert-success");
    }else if(bool==2){
        warning.textContent="Datos actualizados correctamente, " + name;
        warning.setAttribute("class", "warning alert alert-success");
    }

    showBoxAux(warning);
};

////////////////////////////////////

function showBoxSuccessMember(bool){
    const warning = document.createElement("div");

    if(bool==1){
        warning.textContent="Suscripción comprada correctamente.";
        warning.setAttribute("class", "warning alert alert-success");
    }else if(bool==2){
        warning.textContent="Suscripción extendida correctamente.";
        warning.setAttribute("class", "warning alert alert-success");
    }else if(bool==-1){
        warning.textContent="Error al comprar la suscripción.";
        warning.setAttribute("class", "warning alert alert-danger");
    }else if(bool==-2){
        warning.textContent="Error al extender la suscripción";
        warning.setAttribute("class", "warning alert alert-danger");
    }

    showBoxAux(warning);
};

```

Extracto del fichero mencionado mostrando dos funciones: La primera se ejecuta en el index de la página, avisando de inicio de sesión, cierre de sesión o modificación correcta de datos. La segunda se ejecuta en la sección de gestión de alquileres, mostrando mensajes de éxito o error al realizar un pago o una extensión de alquiler

```

function showBoxAux(box){
    document.body.appendChild(box);

    setTimeout(function() { box.classList.add("show-warning"); }, 10);
    setTimeout(function() { box.classList.remove("show-warning"); }, 2000);
    setTimeout(function() { box.style.display = "none"; }, 2500);
}

```

Función de ocultación del mensaje

```
.warning {
  position: fixed;
  top: -50px;
  left: 50%;
  transform: translateX(-50%);
  padding: 10px;
  margin-top: 15px;
  font-size: 16px;
  border-radius: 5px;
  z-index: 1000;
  width: 80%;
  text-align: center;
  transition: top 0.5s ease-in-out;
}
.show-warning { top: 0; }
```

Estilos necesarios para el correcto funcionamiento de los avisos



Ejemplo de funcionamiento: Inicio de Sesión

7. MANUAL DE USO

7.1 REQUISITOS E INSTALACIÓN

Para instalación de esta aplicación, es necesario un PC con al menos el Sistema Operativo Windows 7, y haber instalado la aplicación XAMPP. Tras esto, moveremos la carpeta del proyecto a C:\xampp\htdocs. Inicializamos XAMPP y activamos Apache y MySQL. Finalmente, abrimos el archivo index.php dentro de la carpeta con el navegador que queramos.

7.2 FUNCIONES USUARIO INVITADO

- **REGISTRARSE:** Crear una cuenta en la aplicación. Como se menciona en apartados anteriores, los datos a rellenar son el E-Mail, CIF, Nombre, Dirección y Teléfono de la empresa, además de la contraseña y (opcional) subir una imagen de perfil. En este caso, sólo se puede registrar como Cliente.
- **INICIAR SESIÓN:** Entrar en la aplicación como usuario, utilizando el Nombre y la contraseña.

Registro de Usuario

Email: *

CIF: *

Nombre: *

Contraseña: *

Teléfono: *

Dirección: *

Avatar: No file selected.

☐ Acepto los Términos y Condiciones y la Política de Privacidad

Los campos marcados con un * son obligatorios

¿Ya tienes una cuenta? [Iniciar Sesión](#)

¿Deseas ser uno de nuestros Proveedores? [Contáctanos](#)

Visualización del formulario de registro

7.3 FUNCIONES USUARIO CLIENTE

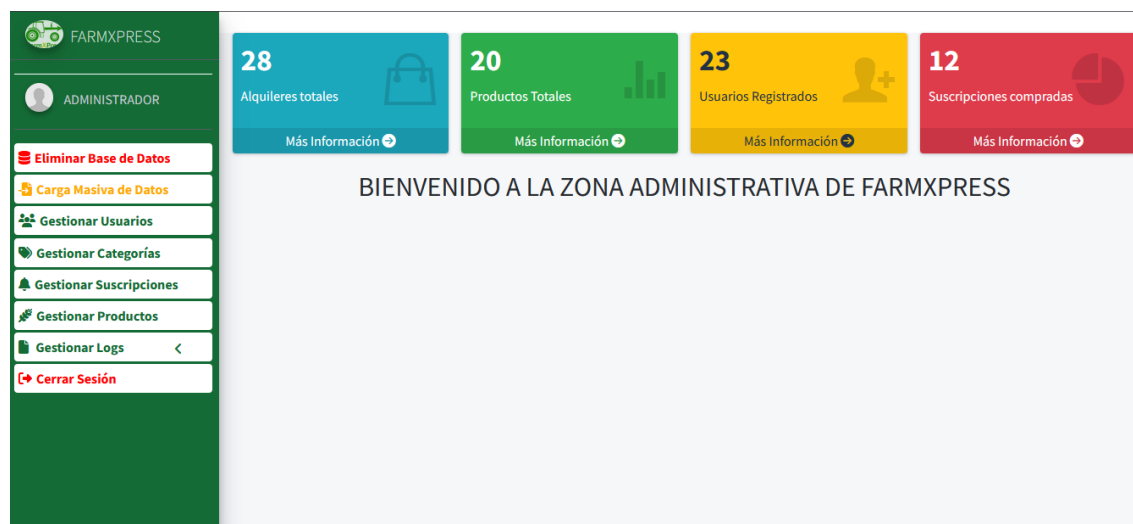
- **ACTUALIZAR DATOS:** Modificar datos de la cuenta, excepto la contraseña.
- **CONSULTAR CATÁLOGO:** Ver el listado de productos existentes en la página. Se puede filtrar por categoría, precio mínimo y precio máximo.
- **ALQUILAR PRODUCTO:** Acceder a la ficha de datos de un producto y, si está disponible, elegir el número de meses durante los que se alquilará el producto.
- **DESCARGAR FACTURA:** Descargar una factura en formato PDF con los datos del alquiler tras realizar el pago inicial mediante Stripe.
- **MARCAR FAVORITOS:** Añadir un producto a la sección de productos favoritos del usuario, para poder realizar un seguimiento más fácil de estos, o eliminarlos. Al pulsar en “Productos Seguidos”, se podrá ver todos estos productos.
- **GESTIONAR ALQUILERES:** Consultar todos los productos que se encuentren alquilados por el usuario en ese momento.
 - **REALIZAR PAGO:** Pagar parcial o totalmente la cantidad debida de un alquiler hasta la fecha, mediante la plataforma de pago Stripe.
 - **EXTENDER ALQUILER:** Junto al pago, opcionalmente se puede extender los meses de duración del alquiler. Si no debes dinero por el alquiler en cuestión, el botón de pago se sustituye por el de “Extender Duración”, el cual elimina el campo de selección de cantidad y la opción de no extenderlo.
 - **FINALIZAR ALQUILER:** Si nuestra deuda es nula, podremos finalizar el alquiler, y el producto volverá a estar disponible para que lo alquile otro usuario.
- **ESCRIBIR RESEÑA DE PRODUCTO:** Si ya hemos alquilado el producto en cuestión al menos una vez, podremos colgar una reseña en su página de detalles. Consiste en un texto y una calificación del 0 al 5, que podremos elegir haciendo click en las estrellas vacías situadas encima del área de texto. Estas estrellas cambiarán de estilo, rellenando su interior para mostrar la calificación marcada.
 - **ELIMINAR RESEÑA:** Podremos eliminar nuestra reseña localizándola y haciendo click en el botón Eliminar. Tras ello, podremos escribir una nueva reseña.
- **ADQUIRIR MEMBRESÍA:** Es posible comprar una de las suscripciones existentes haciendo click en el botón “Comprar” de la tarjeta de la suscripción deseada, y realizando el pago inicial correspondiente al primer mes.
 - **EXTENDER MEMBRESÍA:** Extender los meses de duración de la membresía adquirida.

7.4 FUNCIONES USUARIO PROVEEDOR

- **ACTUALIZAR DATOS:** Modificar datos de la cuenta, excepto la contraseña.
- **GESTIONAR PRODUCTOS:** Ver el listado de los productos que hayamos insertado en la web.
 - **INSERTAR PRODUCTO:** Invocar al formulario de subida de productos en un modal. Se debe rellenar el nombre, descripción, código de referencia y precio mensual de alquiler del producto, y, si se desea, subir una imagen.
 - **ACTUALIZAR PRODUCTO:** Modificar los datos del producto deseado.
 - **ELIMINAR PRODUCTO:** Borrar el producto deseado. Sólo se podrá eliminar si está disponible, es decir, si nadie lo ha alquilado.
- **ADQUIRIR MEMBRESÍA:** Es posible comprar una de las suscripciones existentes haciendo click en el botón “Comprar” de la tarjeta de la suscripción deseada, y realizando el pago inicial correspondiente al primer mes.
 - **EXTENDER MEMBRESÍA:** Extender los meses de duración de la membresía adquirida.

7.5 FUNCIONES ADMINISTRADOR

Para acceder como usuario Administrador, es necesario introducir las siguientes credenciales al iniciar sesión: “ADMINISTRADOR” como nombre de usuario, y “ADMINAPP” como contraseña. Al entrar, veremos este panel de administración:



Las funciones que podemos utilizar pulsando los botones de la barra lateral (o los botones de Más Información) son los siguientes:

- **CREAR LA BASE DE DATOS:** Es posible comprar una de las suscripciones existentes haciendo.
- **ELIMINAR LA BASE DE DATOS:** Es posible comprar una de las suscripciones existentes haciendo.
- **CARGA MASIVA DE DATOS:** Es posible comprar una de las suscripciones existentes haciendo.
- **GESTIONAR USUARIOS:** Ver datos de todos los usuarios en una tabla paginada (10 usuarios por página), y modificar sus datos (botón de engranaje) o eliminarlo (botón de papelera). Al pulsar el botón “Añadir Usuario”, podremos crear manualmente un usuario. Es la única forma de crear Proveedores.
- **GESTIONAR CATEGORÍAS:** Ver información paginada de las diferentes categorías, modificar sus datos, eliminarlas o añadir nuevas.
- **GESTIONAR SUSCRIPCIONES:** Ver información paginada de las diferentes suscripciones, modificar sus datos, eliminarlas o añadir nuevas. Al pulsar el botón de plus, podremos añadir ventajas a la suscripción deseada, y ver el listado de ventajas al pulsar el botón de ojo.

Interfaz de gestión de Suscripciones y Ventajas

- **GESTIONAR PRODUCTOS:** Ver el catálogo de productos. El Administrador puede eliminar los productos, siempre y cuando no se encuentren alquilados.
 - **GESTIONAR RESEÑAS:** Es posible acceder a la ficha de cada producto y eliminar todas las reseñas que el Administrador considere inapropiadas. Por obvias razones, NO es posible realizar un alquiler del producto.
- **GESTIONAR LOGS:** Se ven las siguientes opciones al pulsar “Gestionar Logs”:
 - **VISUALIZAR DATOS:** Podremos ver el tráfico que recibe la web o información de los alquileres realizados. Podemos filtrar la información por usuario, y elegir una fecha de inicio y/o una fecha límite.
 - **IMPRIMIR LOG:** Una vez recibida y mostrada toda la información, es posible crear un fichero TXT con los datos automáticamente al pulsar “Imprimir Informe”. Los ficheros se nombran según el tipo (Visitas o Alquileres) y por fecha de creación.
 - **LEER LOG:** Si hemos creado logs de información con anterioridad, podemos leer los ficheros existentes al pulsar “Consultar Logs Existentes” y elegir el deseado.



Interfaz de visualización de datos

8. CONCLUSIONES

El objetivo de FamXPress es ofrecer una solución eficaz para el alquiler de maquinaria agrícola en España, mediante una plataforma que permita la oferta de los productos por parte de los Proveedores autorizados, debido a la carencia de una página libre y especializada para ello.

La integración de tecnologías como Stripe permite realizar pagos de forma segura, y el uso del patrón de programación MVC permite tener un código estructurado, organizado, escalable y fácilmente mantenible al prevenir el llamado “código espagueti”. Por otro lado, tecnologías front-end como JQuery o AJAX permite el manejo de datos en tiempo real, como el seguimiento de productos, haciendo más cómoda la experiencia para el usuario.

Además, el diseño a partir de una plantilla de Bootstrap ha permitido construir una interfaz amigable y responsiva de forma sencilla, tanto para usuarios comunes como para usuarios administradores.

En resumen, la aplicación propuesta se presenta como una idea con potencial para mejorar y sentar un referente el sector agrícola español, ayudando a empresarios a ofrecer y alquilar maquinaria mediante una web de uso sencillo pero eficaz.

9. PROYECCIÓN A FUTURO

Al ser FamXPress una idea con potencial en el mercado, estos son algunos puntos de mejora que se planea implementar en el futuro:

- **HISTORIAL DE PAGOS:** Implementar un botón que muestre una tabla con el histórico de pagos y extensiones para el alquiler deseado.
- **IMPLEMENTACIÓN DE ENVÍO DE E-MAILS:** No ha sido posible implementar el envío de correos durante el tiempo de desarrollo de la aplicación, pero el envío de correos (verificación de usuario al registrarse o envío de facturas) puede ser un buen añadido.
- **MÉTODOS DE FILTRADO:** Añadir más métodos de filtrado, como nombre del producto o nombre del Proveedor. También la capacidad de ordenar por los diferentes campos en las secciones de gestión del panel de Administración.
- **PÁGINA DE PERFIL:** Sección de perfil del usuario, en el que se muestre su información y, en caso de los Proveedores, todos sus productos.

Se estima que el tiempo de desarrollado de la aplicación es de 4 meses.

10. ENLACES EXTERNOS

- **AGRICULTURE:** <https://bootstrapmade.com/agriculture-bootstrap-website-template/>
- **ADMINLTE:** <https://adminlte.io/>
- **LIBRERÍA PDF24:** <https://www.pdf24.org/es/>
- **DOCUMENTACIÓN STRIPE:** <https://docs.stripe.com/>
- **DOCUMENTACIÓN JQUERY:** <https://api.jquery.com/>
- **GUÍA BOOTSTRAP:** <https://getbootstrap.com/docs/4.1/getting-started/introduction/>
- **RENTING JOHN DEERE :** <https://www.deere.es/es/financiar/financiacion/C3%B3n/leasing-operativo-y-contrato-de-alquiler/>